SQL DATA ANALYSIS PROJECT

Amazon E-Commerce Sales Analysis

A SQL-driven exploration of orders, sellers, customers, revenue and delivery performance on the Amazon platform.

Prepared by: **Samyak Jain**

Project Objective

To analyze Amazon Brazil's e-commerce order data using SQL — covering customers, sellers, products, orders, payments, and reviews — in order to uncover key insights on sales performance, seller efficiency, product trends, delivery timelines, and customer satisfaction to support data-driven business decisions.

Q1. Find the total number of orders fulfilled by each seller state.

ANSWER // SQL QUERY

```
SELECT S.seller_state, COUNT(*)
FROM order_items AS OT
JOIN sellers AS S ON S.seller_id = OT.seller_id
GROUP BY S.seller_state;
```

Q2. For each product category, calculate the cumulative revenue generated as orders come in over time.

ANSWER // SQL QUERY

```
SELECT P.product_category_name,
  ROUND(SUM(OT.price) OVER(
    PARTITION BY P.product_category_name
    ORDER BY O.order_purchase_timestamp
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS Revenue,
  O.order_purchase_timestamp AS Purchase_Time
FROM products AS P
JOIN order_items AS OT ON OT.product_id = P.product_id
JOIN orders AS O ON O.order_id = OT.order_id
WHERE P.product_category_name IS NOT NULL;
```

Q3. Which payment method do customers use the most, and what is the average order value for each payment type?

ANSWER // SQL QUERY

```
WITH Order_Value AS (
  SELECT order_id, SUM(payment_value) AS Total_Amount
  FROM order_payments
  GROUP BY order_id
)
SELECT OP.payment_type, COUNT(DISTINCT OP.order_id),
  ROUND(AVG(OV.Total_Amount), 2) AS Avg_Order_Value
FROM order_payments AS OP
JOIN Order_Value AS OV ON OP.order_id = OV.order_id
GROUP BY OP.payment_type
ORDER BY 3 DESC;
```

Q4. Find the customer who has spent the most money across all their orders.

ANSWER // SQL QUERY

```
SELECT C.customer_id, SUM(OP.payment_value) AS Total_spent
FROM customers AS C
JOIN orders AS O ON O.customer_id = C.customer_id
JOIN order_payments AS OP ON OP.order_id = O.order_id
GROUP BY C.customer_id
ORDER BY Total_spent DESC LIMIT 1;
```

Q5. Find the average review score for each product category.

ANSWER // SQL QUERY

```
SELECT P.product_category_name, ROUND(AVG(R.review_score), 2) AS AVG_Review_Score
FROM products AS P
LEFT JOIN order_items AS OT ON OT.product_id = P.product_id
LEFT JOIN order_reviews AS R ON R.order_id = OT.order_id
GROUP BY P.product_category_name;
```

Q6. Find the total number of orders placed by each customer, broken down by the state they live in.

ANSWER // SQL QUERY

```
SELECT C.customer_id, COUNT(O.order_id) AS Total_Count, C.customer_state
FROM customers AS C
JOIN orders AS O ON O.customer_id = C.customer_id
GROUP BY C.customer_id, C.customer_state
ORDER BY Total_Count DESC;
```

Q7. Identify sellers who registered on the platform but have never fulfilled a single order.

ANSWER // SQL QUERY

```
SELECT S.seller_id, OT.order_item_id
FROM sellers AS S
LEFT JOIN order_items AS OT ON OT.seller_id = S.seller_id
WHERE OT.order_item_id IS NULL;
```

Q8. Find the top 5 product categories by total revenue.

ANSWER // SQL QUERY

```
SELECT P.product_category_name, ROUND(SUM(OI.price), 2) AS Total_Revenue
FROM products AS P
JOIN order_items AS OI ON OI.product_id = P.product_id
GROUP BY P.product_category_name
ORDER BY Total_Revenue DESC LIMIT 5;
```

Q9. Find the median delivery time (in days) between order placement and actual delivery.

ANSWER // SQL QUERY

```
WITH A AS (
  SELECT DATEDIFF(DATE(order_delivered_customer_date), DATE(order_purchase_timestamp)) AS Time_Diff
  FROM orders
),
B AS (
  SELECT Time_Diff, ROW_NUMBER() OVER(ORDER BY Time_Diff) AS RN,
    COUNT(*) OVER() AS Total_Count
  FROM A
)
SELECT AVG(Time_Diff) AS Median_Delivery_Time
FROM B
WHERE RN IN (FLOOR((Total_Count+1)/2),
FLOOR((Total_Count+2)/2));
```

Q10. Find all products that have never been ordered.

ANSWER // SQL QUERY

```
SELECT P.product_category_name, P.product_id
FROM products AS P
LEFT JOIN order_items AS OT ON OT.product_id = P.product_id
WHERE OT.product_id IS NULL;
```

Q11. Find sellers who have fulfilled more orders than the average seller on the platform.

ANSWER // SQL QUERY

```
WITH A AS (
  SELECT S.seller_id, COUNT(DISTINCT OT.order_id) AS Total_Order_Count
  FROM order_items AS OT
  JOIN sellers AS S ON S.seller_id = OT.seller_id
  GROUP BY S.seller_id
)
SELECT seller_id, Total_Order_Count,
  (SELECT AVG(Total_Order_Count) FROM A) AS AVG_Count
FROM A
WHERE Total_Order_Count > (SELECT AVG(Total_Order_Count) FROM A);
```

Q12. Find which Brazilian states have the highest average customer review score for orders delivered there.

ANSWER // SQL QUERY

```
SELECT C.customer_state, ROUND(AVG(R.review_score), 2) AS Avg_Reviews
FROM customers AS C
JOIN orders AS O ON O.customer_id = C.customer_id
JOIN order_reviews AS R ON O.order_id = R.order_id
WHERE LOWER(O.order_status) = "delivered"
GROUP BY C.customer_state
ORDER BY Avg_Reviews DESC LIMIT 1;
```

Q13. Identify customers who have placed orders but never left a review.

ANSWER // SQL QUERY

```
SELECT DISTINCT C.customer_id, R.review_score
FROM customers AS C
JOIN orders AS O ON O.customer_id = C.customer_id
LEFT JOIN order_reviews AS R ON R.order_id = O.order_id
WHERE R.review_id IS NULL;
```

Q14. Find the month with the highest number of orders placed across the entire platform.

ANSWER // SQL QUERY

```
SELECT DATE_FORMAT(order_purchase_timestamp, '%Y-%m') AS Order_month,
  COUNT(*) AS Total_order_placed
FROM orders
GROUP BY DATE_FORMAT(order_purchase_timestamp, '%Y-%m')
ORDER BY Total_order_placed DESC LIMIT 1;
```

THANK YOU

Questions & Discussion — Amazon SQL Analysis Project